Kalshi Weather Market Analysis and Trading Automation

Student: Damian Lapena Vidal
Course: EGR 404
Date: May 2026
GitHub Repository: https://github.com/DamianlapenavidalURI/kalshi-AI-automate

Project Overview

This project developed a local-first automated analysis system for weather-related Kalshi prediction markets. The goal was to create a repeatable workflow that could scan weather markets, gather relevant context, use AI agents to evaluate potential opportunities, and apply deterministic safety checks before any execution decision was allowed.

The main motivation for this project came from the difficulty of manually reviewing many prediction markets. Weather markets can change quickly, and evaluating them requires combining market information, weather context, liquidity, settlement rules, and risk constraints. Manual scanning is slow, inconsistent, and vulnerable to emotional or unsafe decisions. This project attempted to solve that problem by building a structured system where AI supports research and reasoning, but deterministic code controls execution safety.

The final system includes market discovery, weather-focused candidate filtering, multi-agent reasoning, deterministic risk rails, demo execution logic, diagnostics output, SQLite storage, and Streamlit dashboards for both user-level and developer-level visibility. The system does not attempt to give full control to AI. Instead, it uses AI for idea generation, context evaluation, edge reasoning, critique, and final recommendations, while hard-coded safety rules enforce objective restrictions such as market status, liquidity, and repeat-thesis prevention.

Overall, the project successfully produced a working MVP for weather-focused Kalshi market analysis and demo trading automation. The most successful part of the project was the combination of AI-assisted reasoning with deterministic execution safety. The most challenging part was tuning the deterministic settings so the system could be safe without becoming too restrictive.

Problem Addressed

The problem addressed by this project is the difficulty of consistently evaluating short-horizon prediction markets, especially weather-related markets. A human trader or analyst must often answer several questions quickly:

Is the market still open?

Is there enough liquidity?
Is the market settlement condition clear?
Is the weather data reliable enough to support a decision?
Is the system repeating the same thesis too often?
Is there a meaningful edge, or is the market too uncertain?

Doing this manually can be slow and inconsistent. It also creates risk because a person may overlook important details, overtrade similar markets, or make decisions without enough context. Weather markets add another layer of complexity because they depend on changing forecasts, data sources, time horizons, and sometimes unclear settlement conditions.

The project was designed to address these issues by creating a structured workflow that makes market analysis more repeatable and transparent. Instead of relying only on manual judgment, the system separates the process into stages: discovery, filtering, research, AI reasoning, deterministic risk checking, execution, and diagnostics.

The key design principle was that AI should help with reasoning, but not be the only safety mechanism. AI agents can evaluate context and generate recommendations, but final execution must pass deterministic rails before any order intent is accepted.

Project Goals

The project had four main goals.

First, the system needed to support AI-assisted idea generation and critique. This meant using language models to evaluate market context, identify possible edges, criticize weak reasoning, and produce a final decision such as ENTER, SKIP, WAIT, or REDUCE_SIZE.

Second, the system needed deterministic hard rails before execution. The goal was not to let AI freely place trades. Instead, the system needed objective safety rules that could block unsafe or invalid actions regardless of what the AI recommended.

Third, the project needed to be local-first. The system was designed to run from a local development environment, using environment variables, local SQLite storage, local JSON diagnostics, and a Streamlit interface.

Fourth, the system focused on weather-related short-horizon market selection. Although other market categories were considered during development (e.g. soccer games, basketball games), the final system narrowed the scope to weather markets because they provided a clearer project domain and a better fit for structured research and deterministic filtering.

Methodology

The project was developed using an iterative engineering approach. Instead of building the full system all at once, the work was divided into stages.

The first stage was market discovery. The system needed to connect to Kalshi, retrieve candidate markets, and filter them into useful groups. The final implementation focused on several weather-related market families, including hourly temperature, daily temperature, snow and rain, hurricanes, natural disasters, and climate-related markets.

The second stage was candidate filtering. After markets were discovered, the system gathered additional information such as orderbook data, market status, pricing, and other useful metadata. This stage helped determine whether a candidate was worth deeper analysis.

The third stage was research enrichment. The system used weather and external data adapters to support AI reasoning. The project included OpenWeather as a major weather data source, with additional adapters for Open-Meteo, the National Weather Service, the National Hurricane Center, and USGS-related information.

The fourth stage was multi-agent reasoning. Different AI agents were assigned different roles. Some agents performed fast triage, while others evaluated context quality, estimated directional edge, criticized weaknesses, and made a final recommendation.

The fifth stage was deterministic safety checking. Before execution, the system applied hard rails that were not controlled by the AI. These included checking whether the market was open, whether liquidity requirements were met, and whether the same thesis or market had already been repeated too recently.

The sixth stage was execution and diagnostics. The system could create order intents and route them through a deterministic execution engine. It also stored outputs in JSON and SQLite so that results could be reviewed later through dashboards and logs.

Tools and Technologies Used

The project was built primarily in Python 3.11+. Python was used because it supports API integrations, data processing, AI orchestration, database access, and dashboard development in a flexible way.

The main APIs and services used were:

Kalshi API for market discovery, market data, and demo execution logic.
OpenWeather API for weather data.
OpenAI API for AI agent reasoning.
LangChain for AI workflow support and agent orchestration.
Open-Meteo, NWS, NHC, and USGS adapters for additional weather and event context.

The main development tools and libraries were:

Streamlit for the user and developer dashboards.
Altair for visualizations in the dashboard.
SQLite for local persistent storage.
JSON files for diagnostics and latest agent outputs.
Python environment variables and .env configuration for credentials and runtime settings.

The repository also includes scripts, tests, source modules, configuration files, and documentation. The current README explains the main runtime commands, architecture, and system scope.

System Architecture

The final architecture follows a staged pipeline:

Kalshi API and weather/data sources feed the system with market and context data.
Discovery and candidate filtering identify weather markets worth reviewing.
Multi-agent reasoning evaluates the candidate from several perspectives.
A final orchestrator agent produces a final recommendation.
Deterministic risk and execution logic checks whether the recommendation is allowed.
SQLite and JSON diagnostics store the results.
Streamlit dashboards allow the user and developer to inspect system behavior.

The repository documentation describes the unified orchestrator as an explicit staged runtime. Each cycle loads configuration and credentials, builds the Kalshi SDK-backed client, discovers weather candidates, hydrates orderbooks, enriches top candidates with research, runs specialist agents and the final orchestrator, applies hard rails, optionally executes entries, writes diagnostics, and repeats after a polling interval in loop mode.

This architecture was chosen because it separates responsibilities clearly. Market discovery is separate from research. Research is separate from AI reasoning. AI reasoning is separate from execution. Execution is protected by deterministic rules. This separation makes the system easier to debug, safer to run, and easier to extend in the future.

Generative AI Usage

Generative AI was used as a reasoning layer inside the project. The system used different models for different tasks.

The Scout agent performed fast triage. Its purpose was to quickly decide whether a candidate should be kept or rejected, assign a priority score, and provide a short reason.

The Context agent evaluated the quality of the available context. It checked whether the settlement conditions were clear and whether there were concerns or key points that needed attention.

The Edge agent estimated directional edge. It reviewed prices and research context to decide whether there was a possible yes/no opportunity, how confident the system should be, and what evidence supported the idea.

The Critique agent acted as a skeptical reviewer. Its role was to identify risks, apply a confidence penalty if needed, and potentially veto weak opportunities.

The Final Orchestrator agent made the final AI-level decision. It could choose ENTER, SKIP, WAIT, or REDUCE_SIZE, and it provided reasoning, side, and sizing information.

This multi-agent structure was useful because it prevented the system from relying on only one AI output. Instead, the AI workflow included generation, evaluation, criticism, and final synthesis. However, the project intentionally kept AI away from final safety authority. The deterministic rails remained responsible for blocking invalid or unsafe actions.

Deterministic Safety and Risk Rails

A major design choice in this project was to make safety deterministic. This means the system uses hard-coded checks that do not depend on AI judgment.

The most important deterministic rails were:

The market must be open. If a market is closed or unavailable, the system should not attempt to enter.

The market must meet a minimum liquidity requirement. This prevents the system from acting on markets that may be too thin or difficult to trade safely.

The system must avoid repeated market or thesis behavior. If the same market or same idea appears too frequently, the anti-repeat cooldown can block the action.

These rails were important because AI reasoning can be useful but unpredictable. The project's safety model assumes that AI may produce interesting analysis, but execution should only happen when objective conditions are satisfied. This design makes the system more transparent and easier to trust.

Deliverables and Timeline Summary

The project delivered several major components.

Deliverable	Status	Approximate Timeframe
Initial project concept and problem definition	Completed	April 18, 2026

Kalshi market discovery workflow	Completed	April 24, 2026
Weather-focused candidate filtering	Completed	April 24, 2026
Weather/data source adapters	Completed	April 26, 2026
Multi-agent AI reasoning workflow	Completed	April 25, 2026
Final Orchestrator Agent	Completed	April 25, 2026
Deterministic risk rails	Completed	April 24, 2026
Execution path	Completed	May 3, 2026
SQLite diagnostics and JSON outputs	Completed	April 24, 2026
Streamlit user/developer dashboards	Completed	May 2, 2026
GitHub repository documentation	Completed	May 2, 2026
Final presentation	Completed	May 3, 2026
Final report	Completed	May 3, 2026

Some milestones were modified during the project. An earlier soccer and basketball market automation direction was explored but did not become the final focus. That direction involved many markets and was harder to control within the scope of the project. The project was then narrowed to weather markets, which provided a more realistic and structured domain for the final MVP.

Another modified milestone was the level of automation. The project did not aim to create an unrestricted autonomous trading bot. Instead, it became an AI-assisted analysis and demo execution system with deterministic safety rails. This was a better design choice because it made the project safer, more explainable, and more appropriate for an engineering course.

Results and Outcomes

The final project successfully produced a functioning MVP for Kalshi weather market analysis and trading automation.

The system can discover weather-focused candidates from Kalshi, organize them by market family, run AI agents to analyze them, apply deterministic safety rules, and write diagnostics for review. The Streamlit dashboard provides visibility into system behavior, including debugging information and stage timing.

One of the most important outcomes was the confirmation that AI can be useful for research and reasoning, but safety should remain deterministic. The project showed that AI agents are helpful for

evaluating settlement clarity, identifying risks, summarizing context, and comparing market opportunities. However, the project also showed that AI outputs need guardrails because they can be too aggressive, inconsistent, or insufficiently aware of repeated behavior.

The developer dashboard was especially useful because it helped identify why candidates were rejected. Without this visibility, it would be difficult to understand whether the system was failing because of market data, liquidity rules, AI decisions, or deterministic risk settings.

The project also produced a repository that can be run locally, tested through single-run commands, and operated in a continuous loop. The README includes quick-start instructions, main commands, runtime explanation, architecture map, and system scope.

Challenges

The project had several challenges.

One major challenge was finding deterministic settings that were safe but not too restrictive. If the rails were too strict, the system rejected nearly everything. If the rails were too loose, the system could become unsafe or produce low-quality entry intents. Tuning this balance required repeated testing and adjustment.

Another challenge was gathering the expected candidates from Kalshi and dividing them by market type. Weather markets can appear in different forms, and the system needed a way to organize them into useful families such as daily temperature, hourly temperature, hurricanes, climate change, and natural disasters.

A third challenge was finding reliable sources for AI agents to research. The AI reasoning was only as useful as the data and context it received. This required adding adapters and thinking carefully about which sources were relevant for each weather family.

A fourth challenge was controlling repeat-thesis behavior. The system needed to avoid overtrading the same idea or repeatedly acting on highly similar market logic. The anti-repeat cooldown was added to reduce this risk.

A fifth challenge was keeping AI useful without making safety nondeterministic. AI agents can provide flexible reasoning, but they should not be trusted as the only source of risk control. The final architecture solved this by using AI for analysis and deterministic code for hard safety enforcement.

Future Work

Future work should focus on improving reliability, visibility, and evaluation.

The first next step is to continue fine-tuning deterministic settings. The system needs the right balance between safety and usefulness. Future testing should measure how often candidates are rejected, why they are rejected, and whether those rejections are appropriate.

The second next step is to add more automated reporting on outcomes. The system should be able to summarize how many candidates were discovered, how many passed each stage, how many were rejected, and what happened after any demo execution.

The third next step is to improve AI agent behavior visibility. The dashboard could show more detailed agent reasoning, confidence changes, critique results, and final decision logic.

The fourth next step is to improve threat and risk monitoring. The system could track unusual behavior, repeated theses, market volatility, and possible failures in data sources.

The fifth next step is to expand backtesting or historical evaluation. This would help evaluate whether the system's recommendations are useful over time rather than only during live or demo runs.

Conclusion

This project created a working MVP for AI-assisted Kalshi weather market analysis and trading automation. The system addresses the problem of slow and inconsistent manual market scanning by creating a repeatable pipeline for discovery, research, AI reasoning, deterministic risk checking, execution, and diagnostics.

The most important lesson from the project is that AI can be valuable in market analysis, but it should not be the only safety layer. AI is useful for summarizing context, identifying risks, estimating edges, and producing recommendations. However, deterministic code is necessary to enforce objective safety requirements before execution.

The final system demonstrates a strong engineering approach because it emphasizes modular architecture, transparency, local-first operation, diagnostics, and safety.